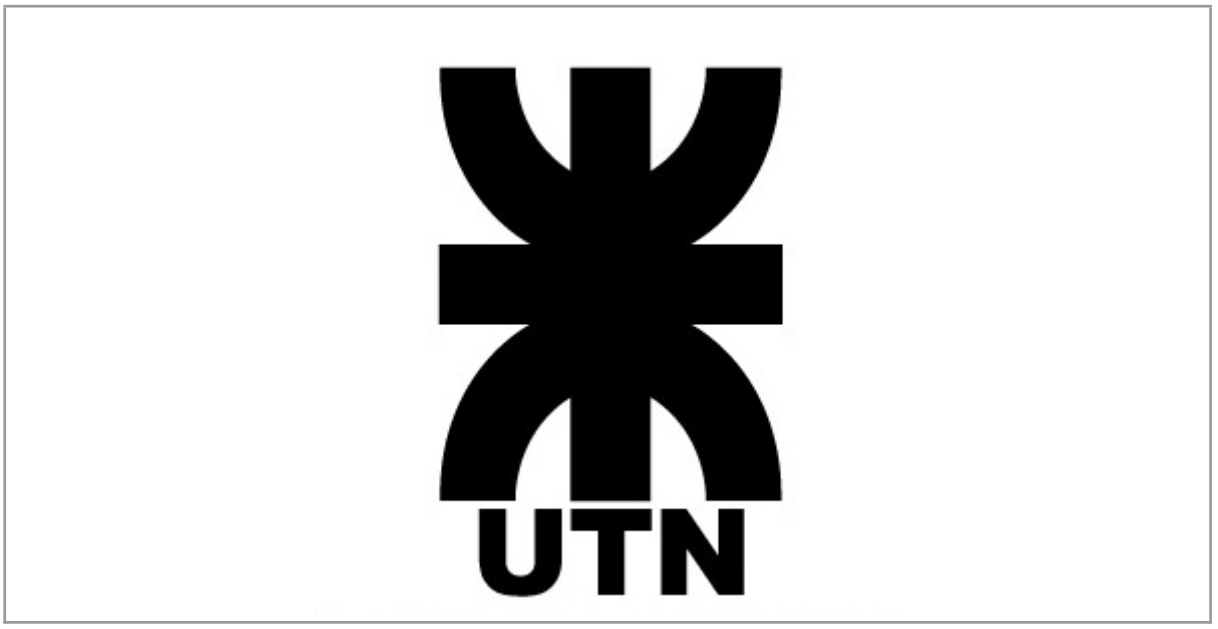
UNIVERSIDAD TECNOLÓGICA NACIONAL

DISEÑO DE SISTEMAS
GRUPO 8



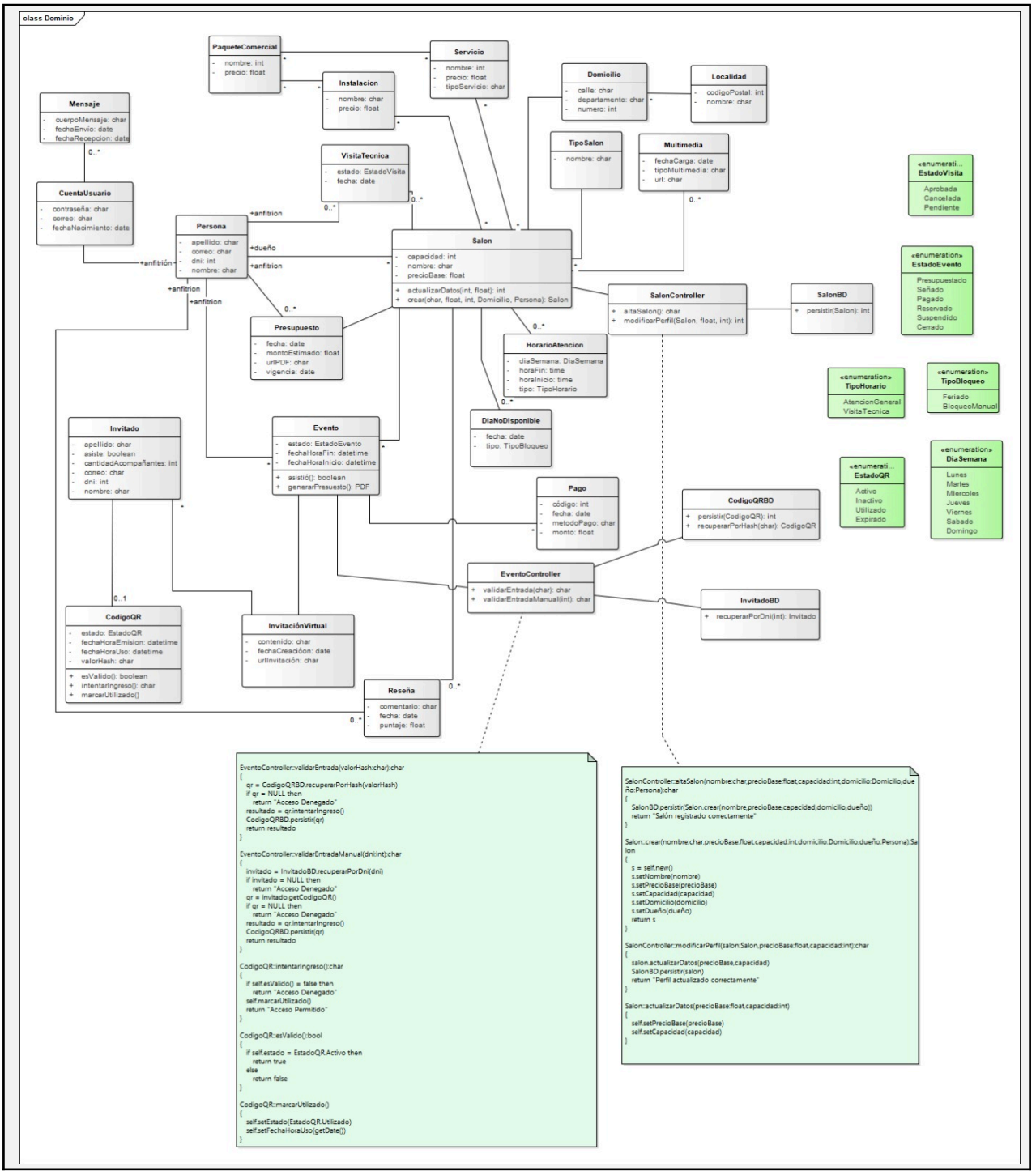
Comisión: 31
Año de cursada: 2026

Integrantes del grupo:

- Mugetti, Emmanuel - Legajo:32534
- Mugetti, Leandro - Legajo: 32535
- Sanchez, Mariano - Legajo: 33079
- Vanucci, Joaquin - Legajo: 33062
- Testa, Daniel - Legajo: 24165

Alcance 3: Eventop

ENTREGA 1: MODELADO DE DOMINIO



RF-001 — Gestión de Perfil del Salón

- **Descripción:** Registrar y modificar datos del perfil: tipo, precio base/hora, ubicación, capacidad, disponibilidad y multimedia.
- **Escenario de prueba:** Modificar el precio base y la capacidad del salón, verificar que los cambios se guarden y actualicen inmediatamente.
- **Finalidad:** Permitir al salón exponer su oferta comercial parametrizada en la plataforma. Objetivo EvenTop: Registro de salones, Encontrar salones por disponibilidad.

Lo resolvimos con SalonController con dos operaciones: altaSalon() para el registro inicial, y modificarPerfil() para editar precio y capacidad después. Usamos Creator: el Controller no arma el objeto Salon a mano, le pide a la propia clase Salon::crear() que se autoconstruya con sus setters — el Controller solo recibe los datos, delega la creación, y después le pide a SalonBD que lo persista. Para la modificación, el mismo criterio: Salon::actualizarDatos() es quien cambia sus propios atributos (Information Expert), y el Controller solo orquesta y persiste.

RF-214 — Control de Acceso

- **Descripción:** Validar la entrada de invitados en tiempo real (manual o vía escaneo QR).
- **Escenario de prueba:** Escanear un código QR válido en la entrada del salón y validar que la pantalla devuelva un mensaje verde de "Acceso Permitido".
- **Finalidad:** Profesionalizar el ingreso al evento, erradicando los fraudes y las largas filas en la puerta.

Lo resolvimos con dos entradas en EventoController: validarEntrada() para el escaneo de QR y validarEntradaManual() para cuando el personal tipea el DNI. Ambos caminos buscan el dato (por hash o por DNI, usando CodigoQRBD/InvitadoBD como Indirection) y delegan toda la decisión a un solo método, CodigoQR::intentarIngreso() si el estado es Activo lo deja pasar y lo marca Utilizado, si no, rechaza. Pusimos la lógica en CodigoQR y no en el Controller porque es quien tiene el dato (estado) para decidir , así evitamos que la misma regla de negocio quede duplicada y desincronizada entre los dos caminos (QR y manual).

----------RF 001----------

```
SalonController::altaSalon(nombre:char,precioBase:float,capacidad:int,domicilio:Domicilio,dueño:Persona):char
{
  SalonBD.persistir(Salon.crear(nombre,precioBase,capacidad,domicilio,dueño))
  return "Salón registrado correctamente"
}

Salon::crear(nombre:char,precioBase:float,capacidad:int,domicilio:Domicilio,dueño:Persona):Salon
{
  s = self.new()
  s.setNombre(nombre)
  s.setPrecioBase(precioBase)
  s.setCapacidad(capacidad)
  s.setDomicilio(domicilio)
  s.setDueño(dueño)
  return s
}

SalonController::modificarPerfil(salon:Salon,precioBase:float,capacidad:int):char
```

```
{
  salon.actualizarDatos(precioBase,capacidad)
  SalonBD.persistir(salon)
  return "Perfil actualizado correctamente"
}

Salon::actualizarDatos(precioBase:float,capacidad:int)
{
  self.setPrecioBase(precioBase)
  self.setCapacidad(capacidad)
}


----------RF214----------


EventoController::validarEntrada(valorHash:char):char
{
  qr = CodigoQRBD.recuperarPorHash(valorHash)
  if qr = NULL then
    return "Acceso Denegado"
  resultado = qr.intentarIngreso()
  CodigoQRBD.persistir(qr)
  return resultado
}

EventoController::validarEntradaManual(dni:int):char
{
  invitado = InvitadoBD.recuperarPorDni(dni)
  if invitado = NULL then
    return "Acceso Denegado"
  qr = invitado.getCodigoQR()
  if qr = NULL then
    return "Acceso Denegado"
  resultado = qr.intentarIngreso()
  CodigoQRBD.persistir(qr)
  return resultado
}

CodigoQR::intentarIngreso():char
{
  if self.esValido() = false then
    return "Acceso Denegado"
  self.marcarUtilizado()
  return "Acceso Permitido"
}

CodigoQR::esValido():bool
{
  if self.estado = EstadoQR.Activo then
    return true
  else
    return false
}
```

```
CodigoQR::marcarUtilizado()
{
  self.setEstado(EstadoQR.Utilizado)
  self.setFechaHoraUso(getDate())
}
```